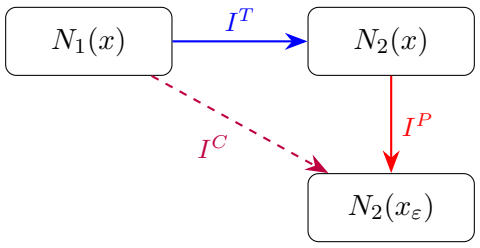
Joining Variables Across Three Network Copies in VHAGaR Transfer Mode

Research Notes

1 Setup and Notation

In VHAGaR transfer mode we encode three network copies[1] sharing the same input $x \in [0,1]^d$ and perturbation $x_\varepsilon = x + e$, $\|e\|_\infty \le \varepsilon$:



where the interval types are:

- **Transfer interval** I^T (blue): bounds the difference $z_\ell^{N_2}(x) - z_\ell^{N_1}(x)$ due to different weights $W_2 \neq W_1$. Currently implemented in `transfer_interval_constraints`.
- **Perturbation interval** I^P (red): bounds the difference $z_\ell^{N_2}(x_\varepsilon) - z_\ell^{N_2}(x)$ due to $\|e\|_\infty \le \varepsilon$. Currently implemented in `perturbed_interval_constraints`.
- **Composed interval** I^C (purple, dashed): bounds $z_\ell^{N_2}(x_\varepsilon) - z_\ell^{N_1}(x)$ directly. *Not yet implemented*—this is the main proposal.

Per-layer notation. For neuron i at layer ℓ (post-activation), denote:

$$a_{\ell,i}^{N_1} = \sigma\big(z_{\ell,i}^{N_1}(x)\big), \qquad a_{\ell,i}^{N_2} = \sigma\big(z_{\ell,i}^{N_2}(x)\big), \qquad \tilde{a}_{\ell,i}^{N_2} = \sigma\big(z_{\ell,i}^{N_2}(x_\varepsilon)\big), \tag{1}$$

where $\sigma(\cdot) = \max(0, \cdot)$ is the ReLU. In the MIP, these correspond to the `x_rect` variables with prefixes `n1_org`, `n2_org`, `n2_pert`.

The existing interval constraints enforce, at every ReLU layer ℓ:

$$a_{\ell,i}^{N_2} \in \big[a_{\ell,i}^{N_1} + I^T_{\ell,i,\text{down}},\ a_{\ell,i}^{N_1} + I^T_{\ell,i,\text{up}}\big], \tag{2}$$

$$\tilde{a}_{\ell,i}^{N_2} \in \big[a_{\ell,i}^{N_2} + I^P_{\ell,i,\text{down}},\ a_{\ell,i}^{N_2} + I^P_{\ell,i,\text{up}}\big]. \tag{3}$$

[1] Four when `n1_p_mode=true`; here we focus on the common case $\{N_1(x),\ N_2(x),\ N_2(x_\varepsilon)\}$.

2 Composed Interval I^C: Full Treatment

2.1 Motivation

The existing interval machinery provides two *pairwise* links:

- I^T: $N_1(x) \longleftrightarrow N_2(x)$ (different weights, same input);
- I^P: $N_2(x) \longleftrightarrow N_2(x_\varepsilon)$ (same weights, different input).

Together they form a *chain* $N_1(x) \to N_2(x) \to N_2(x_\varepsilon)$. Every feasible point in the MIP must satisfy *both* links, yet Gurobi sees them as separate constraint families—it cannot exploit transitivity across them unless we make it explicit.

The composed interval I^C closes this triangle by providing a *direct* bound on the difference $\tilde{a}_\ell^{N_2} - a_\ell^{N_1}$ between the first and last network copies. This adds redundant-but-useful constraints that:

1. Give Gurobi a direct bound between the copies that are farthest apart in the encoding, cutting across the intermediate $N_2(x)$ variables.
2. Enable variable elimination or LP relaxation when the composed interval is tight (Sections 3–5).
3. Are *free* to compute—interval propagation is $O(n^2)$ per layer, negligible compared to MIP solving time.

2.2 Formal Definition

Definition 1 (Composed difference)**.** *For each layer ℓ and neuron i, define the* composed difference

$$\Delta_{\ell,i}^C \overset{def}{=} \tilde{a}_{\ell,i}^{N_2} - a_{\ell,i}^{N_1} \;=\; \sigma\big(z_{\ell,i}^{N_2}(x_\varepsilon)\big) \;-\; \sigma\big(z_{\ell,i}^{N_1}(x)\big). \tag{4}$$

The composed interval *is a pair $I_\ell^C = [I_{\ell,\text{down}}^C,\ I_{\ell,\text{up}}^C]$ such that $\Delta_{\ell,i}^C \in [I_{\ell,i,\text{down}}^C,\ I_{\ell,i,\text{up}}^C]$ for every valid input $x \in [0,1]^d$ and perturbation $\|e\|_\infty \le \varepsilon$.*

2.3 Layer-by-Layer Propagation

We now derive the propagation rules for I^C through each layer type. Let W_1^ℓ, b_1^ℓ and W_2^ℓ, b_2^ℓ be the weights and biases of N_1 and N_2 at layer ℓ, respectively. Let $[\underline{a}_{\ell,i}^{N_1},\ \overline{a}_{\ell,i}^{N_1}]$ denote the activation bounds of $N_1(x)$ at neuron (ℓ, i), stored in `all_bounds_of_original`.

2.3.1 Initialisation (input layer $\ell = 0$)

At the input layer:

$$\Delta_{0,i}^C \;=\; (x_\varepsilon)_i - x_i \;=\; e_i \;\in\; [-\varepsilon,\ \varepsilon]. \tag{5}$$

Therefore:

$$\boxed{I_{0,\text{down}}^C = -\varepsilon \cdot \mathbf{1}, \qquad I_{0,\text{up}}^C = +\varepsilon \cdot \mathbf{1}.} \tag{6}$$

This is *identical* to the initialisation of I^P (the perturbation interval), since at the input the weight difference has not yet taken effect.

2.3.2 Linear layer

At a linear layer ℓ (weights W_1^ℓ, W_2^ℓ, biases b_1^ℓ, b_2^ℓ), the pre-activation values are:

$$z_{\ell,k}^{N_1}(x) = \sum_j (W_1^\ell)_{k,j}\; a_{\ell-1,j}^{N_1} + (b_1^\ell)_k, \tag{7}$$

$$z_{\ell,k}^{N_2}(x_\varepsilon) = \sum_j (W_2^\ell)_{k,j}\; \tilde{a}_{\ell-1,j}^{N_2} + (b_2^\ell)_k. \tag{8}$$

Here we use the convention $(W)_{k,j}$ for the (k,j)-entry of $W^\top$ (i.e. the weight from input j to output k), matching the code's `transpose(l.matrix)`.

Substituting $\tilde{a}_{\ell-1,j}^{N_2} = a_{\ell-1,j}^{N_1} + \Delta_{\ell-1,j}^{C}$:

$$\begin{aligned}
\Delta z_{\ell,k}^{C} &\overset{\text{def}}{=} z_{\ell,k}^{N_2}(x_\varepsilon) - z_{\ell,k}^{N_1}(x) \\
&= \sum_j (W_2^\ell)_{k,j}\left(a_{\ell-1,j}^{N_1} + \Delta_{\ell-1,j}^{C}\right) - \sum_j (W_1^\ell)_{k,j}\, a_{\ell-1,j}^{N_1} + (b_2^\ell - b_1^\ell)_k \\
&= \underbrace{\sum_j \left((W_2^\ell)_{k,j} - (W_1^\ell)_{k,j}\right) a_{\ell-1,j}^{N_1}}_{\text{Term 1: weight-difference applied to } N_1 \text{ activations}} + \underbrace{\sum_j (W_2^\ell)_{k,j}\, \Delta_{\ell-1,j}^{C}}_{\text{Term 2: } W_2 \text{ applied to composed interval}} + (b_2^\ell - b_1^\ell)_k.
\end{aligned} \tag{9}$$

We bound each term by interval arithmetic (denoted IA):

$$[\mathrm{T1}_{\min,k},\ \mathrm{T1}_{\max,k}] = \mathrm{IA}\big((W_2 - W_1)^\ell,\ (W_2 - W_1)^\ell,\ \underline{a}_{\ell-1}^{N_1},\ \overline{a}_{\ell-1}^{N_1}\big), \tag{10}$$

$$[\mathrm{T2}_{\min,k},\ \mathrm{T2}_{\max,k}] = \mathrm{IA}\big((W_2^\ell),\ (W_2^\ell),\ I_{\ell-1,\mathrm{down}}^{C},\ I_{\ell-1,\mathrm{up}}^{C}\big), \tag{11}$$

where $\mathrm{IA}(W_{\mathrm{lo}}, W_{\mathrm{hi}}, v_{\mathrm{lo}}, v_{\mathrm{hi}})$ is the standard interval-matrix-vector product:

$$\mathrm{IA}(W_{\mathrm{lo}}, W_{\mathrm{hi}}, v_{\mathrm{lo}}, v_{\mathrm{hi}})_k = \sum_j \big[\min(W_{\mathrm{lo},k,j} \cdot v_{\mathrm{lo},j},\ W_{\mathrm{lo},k,j} \cdot v_{\mathrm{hi},j},\ W_{\mathrm{hi},k,j} \cdot v_{\mathrm{lo},j},\ W_{\mathrm{hi},k,j} \cdot v_{\mathrm{hi},j})\big]. \tag{12}$$

(This is exactly `interval_matrix_vector_multiplication` in the codebase. When $W_{\mathrm{lo}} = W_{\mathrm{hi}} = W$, it simplifies to standard interval–vector multiplication with a fixed matrix.)

The pre-activation composed interval is then:

$$\boxed{I_{\ell,k,\mathrm{down}}^{C,(z)} = \mathrm{T1}_{\min,k} + \mathrm{T2}_{\min,k} + (b_2 - b_1)_k, \qquad I_{\ell,k,\mathrm{up}}^{C,(z)} = \mathrm{T1}_{\max,k} + \mathrm{T2}_{\max,k} + (b_2 - b_1)_k.} \tag{13}$$

Remark 1 (Comparison with existing propagation formulas)**.** *The transfer interval I^T uses the same Term 1 but replaces Term 2 with* $\mathrm{IA}(W_1, W_1, I_{\mathrm{down}}^T, I_{\mathrm{up}}^T)$*—i.e. the* reference *network weights W_1 applied to the transfer interval. The perturbation interval I^P has no Term 1 (since both copies use the same weights) and uses* $\mathrm{IA}(W_2, W_2, I_{\mathrm{down}}^P, I_{\mathrm{up}}^P)$*. The composed I^C "sees" both effects simultaneously through a single propagation, using the correct matrix W_2 in Term 2 (the network that actually processes x_ε) and $(W_2 - W_1)$ in Term 1.*

2.3.3 ReLU layer

At a ReLU layer, we need to bound the post-activation difference:

$$\Delta^C_{\ell,i} = \sigma(z^{N_2}_{\ell,i}(x_\varepsilon)) - \sigma(z^{N_1}_{\ell,i}(x)). \tag{14}$$

Given that the pre-activation difference satisfies $\Delta z^C_{\ell,i} \in [I^{C,(z)}_{\ell,i,\text{down}},\ I^{C,(z)}_{\ell,i,\text{up}}]$, we apply the following relaxation:

Proposition 1 (ReLU difference relaxation)**.** *For any $z_1, z_2 \in \mathbb{R}$ with $z_2 - z_1 \in [\delta_{\min}, \delta_{\max}]$:*

$$\sigma(z_2) - \sigma(z_1) \ \in\ \big[\min(0,\ \delta_{\min}),\ \ \max(0,\ \delta_{\max})\big]. \tag{15}$$

Proof. We verify (15) by case analysis on the signs of z_1, z_2:

Case 1: $z_1 \geq 0$ and $z_2 \geq 0$. Then $\sigma(z_2) - \sigma(z_1) = z_2 - z_1 \in [\delta_{\min}, \delta_{\max}] \subseteq [\min(0, \delta_{\min}), \max(0, \delta_{\max})]$. ✓

Case 2: $z_1 \leq 0$ and $z_2 \leq 0$. Then $\sigma(z_2) - \sigma(z_1) = 0 \in [\min(0, \delta_{\min}), \max(0, \delta_{\max})]$ since the interval always contains 0. ✓

Case 3: $z_1 \geq 0$ and $z_2 \leq 0$. Then $\sigma(z_2) - \sigma(z_1) = -z_1 \leq 0$. Since $z_2 \leq 0 \leq z_1$, we have $\delta_{\min} \leq z_2 - z_1 \leq -z_1 \leq 0$, so $-z_1 \geq z_2 - z_1 \geq \delta_{\min}$, giving $-z_1 \geq \min(0, \delta_{\min})$. Also $-z_1 \leq 0 \leq \max(0, \delta_{\max})$. ✓

Case 4: $z_1 \leq 0$ and $z_2 \geq 0$. Then $\sigma(z_2) - \sigma(z_1) = z_2 \geq 0$. Since $z_2 = z_1 + (z_2 - z_1) \leq 0 + \delta_{\max} = \delta_{\max}$, we get $z_2 \leq \max(0, \delta_{\max})$. Also $z_2 \geq 0 \geq \min(0, \delta_{\min})$. ✓ □

Therefore the post-activation composed interval is:

$$\boxed{I^C_{\ell,i,\text{down}} = \min\big(0,\ I^{C,(z)}_{\ell,i,\text{down}}\big), \qquad I^C_{\ell,i,\text{up}} = \max\big(0,\ I^{C,(z)}_{\ell,i,\text{up}}\big).} \tag{16}$$

The MIP constraint added at each ReLU layer is:

$$a^{N_1}_{\ell,i} + I^C_{\ell,i,\text{down}} \ \leq\ \tilde{a}^{N_2}_{\ell,i} \ \leq\ a^{N_1}_{\ell,i} + I^C_{\ell,i,\text{up}}. \tag{17}$$

2.3.4 Flatten layer

Flatten is a permutation/reshape with no arithmetic. The composed interval is simply reshaped:

$$I^C_{\text{post-flatten}} = \texttt{flatten}(I^C_{\text{pre-flatten}}). \tag{18}$$

2.4 Complete Propagation Algorithm

We summarise the full propagation in one place:

Algorithm 1 Propagation of composed interval I^C through an L-layer network

Require: Networks $N_1 = (W_1^\ell, b_1^\ell)_{\ell=1}^L$, $N_2 = (W_2^\ell, b_2^\ell)_{\ell=1}^L$; perturbation budget ε; activation bounds $[\underline{a}_\ell^{N_1}, \overline{a}_\ell^{N_1}]$ for each layer

Ensure: Per-layer composed intervals I_ℓ^C for $\ell = 0, \ldots, L$

$I_{0,\text{up}}^C \leftarrow +\varepsilon \cdot \mathbf{1}$; $\quad I_{0,\text{down}}^C \leftarrow -\varepsilon \cdot \mathbf{1}$ ▷ Eq. (6)

for $\ell = 1, \ldots, L$ **do**

  if layer ℓ is FLATTEN **then**

    $I_\ell^C \leftarrow \texttt{flatten}(I_{\ell-1}^C)$

  else if layer ℓ is LINEAR **then**

    $\Delta W \leftarrow (W_2^\ell - W_1^\ell)^\top$ ▷ Weight-difference matrix

    $[\text{T1}_{\min}, \text{T1}_{\max}] \leftarrow \text{IA}(\Delta W, \Delta W, \underline{a}_{\ell-1}^{N_1}, \overline{a}_{\ell-1}^{N_1})$ ▷ Eq. (10)

    $[\text{T2}_{\min}, \text{T2}_{\max}] \leftarrow \text{IA}((W_2^\ell)^\top, (W_2^\ell)^\top, I_{\ell-1,\text{down}}^C, I_{\ell-1,\text{up}}^C)$ ▷ Eq. (11)

    $I_{\ell,\text{down}}^{C,(z)} \leftarrow \text{T1}_{\min} + \text{T2}_{\min} + (b_2^\ell - b_1^\ell)$

    $I_{\ell,\text{up}}^{C,(z)} \leftarrow \text{T1}_{\max} + \text{T2}_{\max} + (b_2^\ell - b_1^\ell)$ ▷ Eq. (13)

    $I_\ell^C \leftarrow I_\ell^{C,(z)}$ ▷ Store pre-activation; ReLU applied next

  else if layer ℓ is RELU **then**

    $I_{\ell,\text{up}}^C \leftarrow \max(0,\ I_{\ell,\text{up}}^C)$ ▷ Eq. (16)

    $I_{\ell,\text{down}}^C \leftarrow -\max(0,\ -I_{\ell,\text{down}}^C)$

    Add MIP constraint (17) at this layer

  end if

end for

2.5 Soundness

Definition 2 (Soundness)**.** *An interval constraint $\tilde{a}_{\ell,i}^{N_2} \in [a_{\ell,i}^{N_1} + L_i,\ a_{\ell,i}^{N_1} + U_i]$ is* sound *if every pair (x, e) with $x \in [0,1]^d$ and $\|e\|_\infty \le \varepsilon$ satisfies the constraint. Equivalently: the true feasible set (all points satisfying the exact network equations) is contained within the MIP feasible set augmented with these constraints. Adding sound constraints to a maximisation MIP never decreases the optimal value—it can only tighten the relaxation.*

Proposition 2 (Soundness of I^C)**.** *The composed interval I_ℓ^C computed by Algorithm 1 is sound at every layer ℓ: for all $x \in [0,1]^d$ and $\|e\|_\infty \le \varepsilon$,*

$$\tilde{a}_{\ell,i}^{N_2}(x_\varepsilon) - a_{\ell,i}^{N_1}(x) \ \in\ [I_{\ell,i,\text{down}}^C,\ I_{\ell,i,\text{up}}^C]. \tag{19}$$

Proof. By induction on the layer index ℓ.

Base case ($\ell = 0$). $\tilde{a}_{0,i}^{N_2} - a_{0,i}^{N_1} = (x+e)_i - x_i = e_i \in [-\varepsilon, \varepsilon] = [I_{0,\text{down},i}^C, I_{0,\text{up},i}^C]$. ✓

Inductive step (Linear layer ℓ). Assume $\Delta_{\ell-1,j}^C \in [I_{\ell-1,j,\text{down}}^C, I_{\ell-1,j,\text{up}}^C]$ for all j (inductive hypothesis), and $a_{\ell-1,j}^{N_1} \in [\underline{a}_{\ell-1,j}^{N_1}, \overline{a}_{\ell-1,j}^{N_1}]$ (from the MIP encoding of N_1, which computes valid bounds during bound tightening).

From (9), the pre-activation difference is a sum of products of bounded quantities. The interval arithmetic computation (10)–(11) is the standard outer-approximation of a bilinear form over a box domain, which is exact for each individual product (the min/max of four products covers all sign combinations). Summing these per-element bounds gives a valid enclosure for the full sum. Adding the constant bias difference preserves validity. Therefore $\Delta z_{\ell,k}^C \in [I_{\ell,k,\text{down}}^{C,(z)}, I_{\ell,k,\text{up}}^{C,(z)}]$. ✓

Inductive step (ReLU layer ℓ). Given $\Delta z^C_{\ell,i} \in [I^{C,(z)}_{\ell,i,\text{down}}, I^{C,(z)}_{\ell,i,\text{up}}]$ from the preceding linear layer, Proposition 1 gives $\Delta^C_{\ell,i} = \sigma(z^{N_2}_{\ell,i}) - \sigma(z^{N_1}_{\ell,i}) \in [\min(0, I^{C,(z)}_{\text{down}}), \max(0, I^{C,(z)}_{\text{up}})] = [I^C_{\ell,i,\text{down}}, I^C_{\ell,i,\text{up}}]$. ✓

Flatten layer. Flatten is a bijective reshape; it does not change any values, only their indexing. The interval is reshaped identically. ✓

By induction, soundness holds at every layer. □

Remark 2 (What soundness means for the transfer proof)**.** *The VHAGaR transfer MIP is a* maximisation *problem (*max δ_{diff}*). Adding the sound constraints* (17) *can only* reduce *the feasible set (or leave it unchanged), which means the optimal value can only decrease (or stay the same). Thus:*

- *If the MIP with I^C constraints finds $\delta^*_{\text{diff}} \geq 0$ (i.e. a feasible point exists), that point is also feasible for the original MIP—the* primal bound *(incumbent) is valid.*
- *The* dual bound *(best bound / relaxation value) can only improve (decrease towards the true optimum), because the feasible region is tighter.*
- *The proof is never invalidated: I^C constraints are additional valid inequalities, just like cuts in branch-and-cut.*

2.6 Completeness (Tightness of the Relaxation)

Soundness guarantees no false negatives (no valid point is excluded). Completeness asks about false positives: does the interval ever include values that cannot be realised?

Proposition 3 (Tightness at initialisation)**.** *At the input layer, $I^C_0 = [-\varepsilon, \varepsilon]^d$ is* tight*: for every value $v \in [-\varepsilon, \varepsilon]^d$, there exists (x, e) with $x \in [0,1]^d$, $\|e\|_\infty \leq \varepsilon$, and $e = v$.*

Proof. Choose $x_i = \text{clip}(0.5, \varepsilon, 1-\varepsilon)$ (so that $x_i + v_i \in [0,1]$) and $e_i = v_i$. Since $|v_i| \leq \varepsilon$, all constraints are satisfied. □

Proposition 4 (Tightness of the linear step)**.** *The interval arithmetic in* (10)–(11) *is* tight for each summand*: for each (k, j), the min and max of $W_{k,j} \cdot v_j$ over $v_j \in [v_{\min,j}, v_{\max,j}]$ are achieved exactly. The sum over j may introduce over-approximation due to the "dependency problem" of interval arithmetic (the same variable $a^{N_1}_{\ell-1,j}$ appears in multiple terms but is treated independently).*

Proposition 5 (Over-approximation at ReLU)**.** *The ReLU relaxation* (15) *is* not tight *in general. The gap arises because we only track the* difference $\Delta z = z_2 - z_1$*, not the individual values z_1 and z_2.*

Specifically, the value $\max(0, \delta_{\max})$ *is achievable only when $z_2 \geq 0$ (so that $\sigma(z_2) = z_2$) and $z_1 \leq 0$ (so that $\sigma(z_1) = 0$). If the individual bounds on z_1, z_2 rule this out—for instance, if $z_1 > 0$ always—then the true range is tighter than* $[\min(0, \delta_{\min}), \max(0, \delta_{\max})]$.

Remark 3 (Completeness is a spectrum)**.** *In neural network verification, interval propagation is always an* outer approximation*. The composed interval I^C has the same completeness status as the existing I^T and I^P: sound but not tight in general. The over-approximation grows with network depth, which is inherent to any interval-arithmetic approach. What matters in practice is that I^C provides tighter constraints than what Gurobi can infer on its own from the separate I^T and I^P constraints.*

2.7 Comparison with the Naïve Sum $I^T + I^P$

A simpler alternative to direct propagation is to compute I^T and I^P independently (as the codebase already does) and take their Minkowski sum at each layer:

$$I^{C,\text{naive}}_{\ell,\text{down}} = I^T_{\ell,\text{down}} + I^P_{\ell,\text{down}}, \qquad I^{C,\text{naive}}_{\ell,\text{up}} = I^T_{\ell,\text{up}} + I^P_{\ell,\text{up}}. \tag{20}$$

This is the approach stated in Proposition 6 below. We show that direct propagation is *strictly tighter*.

Proposition 6 (Naïve sum is sound)**.** *The naïve sum* (20) *satisfies* $\Delta^C_\ell \in [I^{C,naive}_{\ell,down}, I^{C,naive}_{\ell,up}]$ *for all valid inputs.*

Proof. $\Delta^C_\ell = (\tilde{a}^{N_2}_\ell - a^{N_2}_\ell) + (a^{N_2}_\ell - a^{N_1}_\ell)$. The first term lies in $[I^P_{\text{down}}, I^P_{\text{up}}]$ and the second in $[I^T_{\text{down}}, I^T_{\text{up}}]$. By interval addition, the sum lies in $[I^T_{\text{down}} + I^P_{\text{down}}, I^T_{\text{up}} + I^P_{\text{up}}]$. □

Proposition 7 (Direct I^C is at least as tight as naïve sum)**.** *At every layer* ℓ*:*

$$I^C_{\ell,\text{down}} \;\geq\; I^{C,\text{naive}}_{\ell,\text{down}}, \qquad I^C_{\ell,\text{up}} \;\leq\; I^{C,\text{naive}}_{\ell,\text{up}}. \tag{21}$$

That is, the directly-propagated interval is contained within the naïve sum. The containment can be strict *at ReLU layers.*

Proof. At the input layer ($\ell = 0$), both are $[-\varepsilon, \varepsilon]$—equal.

At linear layers, the direct and naïve propagations would give the same width *if* the inputs had the same width. Since at the previous ReLU the direct interval may be tighter (shown below), the linear step preserves or amplifies any width advantage. (Linear operations are monotone in interval width.)

The key difference is at **ReLU layers**. Consider the scalar case for clarity. Let the pre-activation transfer interval be $[a, b]$ (so $I^T_{\text{down}} = a$, $I^T_{\text{up}} = b$) and the pre-activation perturbation interval be $[c, d]$ ($I^P_{\text{down}} = c$, $I^P_{\text{up}} = d$).

Naïve sum (two separate ReLU relaxations, then sum):

$$I^T_{\text{post-ReLU}} = [\min(0, a),\ \max(0, b)], \tag{22}$$

$$I^P_{\text{post-ReLU}} = [\min(0, c),\ \max(0, d)], \tag{23}$$

$$I^{C,\text{naive}} = [\min(0, a) + \min(0, c),\ \ \max(0, b) + \max(0, d)]. \tag{24}$$

Direct I^C (sum first, then one ReLU relaxation):

$$I^C = [\min(0,\ a + c),\ \ \max(0,\ b + d)]. \tag{25}$$

Now compare the upper bounds:

$$\max(0,\ b + d) \;\leq\; \max(0, b) + \max(0, d) \tag{26}$$

because $\max(0, \cdot)$ is *subadditive*: $\max(0, x + y) \leq \max(0, x) + \max(0, y)$ for all $x, y \in \mathbb{R}$ (since if $x + y > 0$ and, say, $x > 0, y \leq 0$, then $\max(0, x + y) = x + y < x = \max(0, x) + \max(0, y)$).

Similarly, for the lower bounds:

$$\min(0,\ a + c) \;\geq\; \min(0, a) + \min(0, c) \tag{27}$$

because $\min(0, \cdot)$ is *superadditive*.

Therefore $[I^C_{\text{down}}, I^C_{\text{up}}] \subseteq [I^{C,\text{naive}}_{\text{down}}, I^{C,\text{naive}}_{\text{up}}]$.

Strict improvement. The inequality in (26) is strict when $b > 0$ and $d < 0$ (or vice versa): partial cancellation between the transfer and perturbation effects reduces the composed upper bound, but the naïve sum cannot capture this because it applies $\max(0, \cdot)$ to each interval independently before summing.

Concretely: if $b = 0.3$ and $d = -0.1$, then:

$$\begin{aligned} \text{Naïve:} \quad & \max(0, 0.3) + \max(0, -0.1) = 0.3 + 0 = 0.3, \\ \text{Direct:} \quad & \max(0, 0.3 + (-0.1)) = \max(0, 0.2) = 0.2. \end{aligned}$$

The direct propagation is tighter by 0.1. □

Remark 4 (When does I^C help most?)**.** *The improvement is largest when:*

1. *The transfer and perturbation intervals have* opposite signs *at many neurons (partial cancellation at ReLU).*
2. *The network is deep (more ReLU layers ⇒ more opportunities for cumulative tightening).*
3. *$N_1 \approx N_2$ (similar weights) so that I^T is small and the composed interval stays tight through many layers.*

2.8 Relationship to Existing Code

The composed interval I^C can be implemented as a third call alongside the existing `transfer_interval_constrain` and `perturbed_interval_constraints`, following the same code pattern. It adds constraints between the `n1_org` and `n2_pert` prefixes (which currently have no direct interval link).

The three constraint families are *complementary*:

Constraint	**Links**	**Prefix pair**
I^T (`transfer_interval_constraints`)	$N_1(x) \leftrightarrow N_2(x)$	`n1_org` ↔ `n2_org`
I^P (`perturbed_interval_constraints`)	$N_2(x) \leftrightarrow N_2(x_\varepsilon)$	`n2_org` ↔ `n2_pert`
I^C (**new**)	$N_1(x) \leftrightarrow N_2(x_\varepsilon)$	`n1_org` ↔ `n2_pert`

All three can be active simultaneously without conflict—they are all sound (valid inequalities) and constrain different variable pairs. When all three are present, the LP relaxation at each Gurobi node is tighter than with any two, because the solver can use I^C to directly prune the search space of $N_2(x_\varepsilon)$ without reasoning through $N_2(x)$ as an intermediate.

3 Idea 2: Variable Merging via Interval Width Threshold

Inspired by LUCID's `me_th` mechanism: when the interval gap at neuron (ℓ, i) is below a threshold θ, we can *eliminate* the corresponding variable from one or more copies.

Definition 3 (Interval width)**.** *For each neuron (ℓ, i), define the interval widths:*

$$\gamma^T_{\ell,i} = I^T_{\ell,i,up} - I^T_{\ell,i,down}, \quad \textit{(transfer width)} \tag{28}$$

$$\gamma^P_{\ell,i} = I^P_{\ell,i,up} - I^P_{\ell,i,down}, \quad \textit{(perturbation width)} \tag{29}$$

$$\gamma^C_{\ell,i} = I^C_{\ell,i,up} - I^C_{\ell,i,down} = \gamma^T_{\ell,i} + \gamma^P_{\ell,i}. \quad \textit{(composed width)} \tag{30}$$

3.1 Merging $N_2(x)$ into $N_1(x)$ when γ^T is small

When $\gamma^T_{\ell,i} < \theta$ for neuron (ℓ, i):

1. Remove the separate `n2_org` variable for this neuron.
2. Replace all references to $a^{N_2}_{\ell,i}$ with $a^{N_1}_{\ell,i} + \bar{I}^T_{\ell,i}$, where $\bar{I}^T = \frac{1}{2}(I^T_{\text{up}} + I^T_{\text{down}})$ is the interval midpoint.
3. This eliminates one continuous variable and one binary variable per merged neuron.

Effect on the MIP. Each merged neuron saves:

- 1 continuous variable (`x_rect`)
- 1 binary variable (`a`) if the neuron is split
- 4 big-M constraints (the ReLU encoding)

The trade-off: we introduce an approximation error of at most $\frac{1}{2}\gamma^T_{\ell,i}$.

3.2 Merging $N_2(x_\varepsilon)$ into $N_2(x)$ when γ^P is small

Analogously, when $\gamma^P_{\ell,i} < \theta$:

1. Remove the `n2_pert` variable.
2. Replace with $a^{N_2}_{\ell,i} + \bar{I}^P_{\ell,i}$.

3.3 Full three-way merge when γ^C is small

When $\gamma^C_{\ell,i} < \theta$, *both* $N_2(x)$ and $N_2(x_\varepsilon)$ can be expressed in terms of $N_1(x)$:

$$a^{N_2}_{\ell,i} \approx a^{N_1}_{\ell,i} + \bar{I}^T_{\ell,i}, \tag{31}$$

$$\tilde{a}^{N_2}_{\ell,i} \approx a^{N_1}_{\ell,i} + \bar{I}^C_{\ell,i}. \tag{32}$$

This eliminates **two** sets of variables per neuron (both `n2_org` and `n2_pert`), replacing them with affine expressions of the $N_1(x)$ variable plus a constant offset.

4 Idea 3: Selective Variable Sharing (Exact, Not Approximate)

The merging in Section 3 introduces approximation error. Here we describe an *exact* approach that still reduces variables.

4.1 Equality from dependencies ($\phi_{\text{dep}} = 0$)

From VHAGaR's dependency propagation, when $\phi_{\text{dep},\ell,i} = 0$ for the $N_1(x) \leftrightarrow N_2(x)$ pair (i.e., the perturbation type maps this neuron identically), we already know:

$$z^{N_1}_{\ell,i}(x) = z^{N_2}_{\ell,i}(x) \quad \Longrightarrow \quad a^{N_1}_{\ell,i} = a^{N_2}_{\ell,i}.$$

Currently, `encode_dependencies` adds an *equality constraint* `av[ind_o+1] == av[ind_p+1]`, but both variables still exist.

Proposal: Instead of adding the constraint, make $N_2(x)$ *reuse* the $N_1(x)$ variable during encoding. When $\phi_{\text{dep}} = 0$:

1. During `v_in |> nn2` (the $N_2(x)$ forward pass), substitute the already-created `n1_org` variable for this neuron.
2. Skip creating a new `n2_org` variable, its binary, and its big-M constraints.
3. The constraint is satisfied by construction.

This is **exact**—no approximation. The savings are the same as Section 3 but with zero error.

4.2 Equality from zero intervals ($I^T = [0, 0]$)

If the transfer interval propagation yields $I^T_{\ell,i,\text{up}} = I^T_{\ell,i,\text{down}} = 0$ (which happens when $W_1 = W_2$ through the path to this neuron), then again $a^{N_2}_{\ell,i} = a^{N_1}_{\ell,i}$ exactly, and the same reuse strategy applies.

4.3 Combined condition

In practice, both transfer and perturbation intervals can independently yield exact equalities. The full set of sharing conditions:

Condition	$N_2(x)$ **var**	$N_2(x_\varepsilon)$ **var**	**Savings**
$I^T_{\ell,i} = [0, 0]$	reuse $N_1(x)$	keep	1 cont + 1 bin
$I^P_{\ell,i} = [0, 0]$	keep	reuse $N_2(x)$	1 cont + 1 bin
$I^T = I^P = [0, 0]$	reuse $N_1(x)$	reuse $N_1(x)$	2 cont + 2 bin
$\phi_{\text{dep}} = 0$ *and* $I^P = [0, 0]$	reuse $N_1(x)$	reuse $N_1(x)$	2 cont + 2 bin

5 Idea 4: Hybrid LP Relaxation + Variable Reuse

Combining the LUCID threshold idea with the exact reuse:

Algorithm 2 Hybrid variable management for neuron (ℓ, i)

Require: Interval widths $\gamma^T_{\ell,i}$, $\gamma^P_{\ell,i}$; dependency $\phi_{\ell,i}$; threshold θ; bounds $[l_1, u_1]$ for $N_1(x)$

// **Phase 1: Encode** $N_1(x)$ — **always full big-M**
Create $a^{N_1}_{\ell,i}$ with binary α^{N_1} and 4 constraints

// **Phase 2: Encode** $N_2(x)$
if $\gamma^T_{\ell,i} = 0$ **or** $\phi_{\ell,i} = 0$ **then**
    $a^{N_2}_{\ell,i} \leftarrow a^{N_1}_{\ell,i}$ ▷ Exact reuse—no new variables
else if $\gamma^T_{\ell,i} < \theta$ **then**
    Create $a^{N_2}_{\ell,i}$ as continuous only (LP relaxation, no binary)
    Add interval constraints (2) linking to a^{N_1}
else
    Create $a^{N_2}_{\ell,i}$ with full big-M encoding
    Add interval constraints (2)
end if

// **Phase 3: Encode** $N_2(x_\varepsilon)$
if $\gamma^P_{\ell,i} = 0$ **then**
    $\tilde{a}^{N_2}_{\ell,i} \leftarrow a^{N_2}_{\ell,i}$ ▷ Exact reuse
else if $\gamma^P_{\ell,i} < \theta$ **then**
    Create $\tilde{a}^{N_2}_{\ell,i}$ as continuous only (LP relaxation)
    Add interval constraints (3) linking to a^{N_2}
else
    Create $\tilde{a}^{N_2}_{\ell,i}$ with full big-M encoding
    Add interval constraints (3)
end if

// **Phase 4: Add composed constraint (bonus tightening)**
if a^{N_2} was *not* reused from N_1 and $\gamma^C < \theta_C$ **then**
    Add composed interval constraints (17) ▷ Extra link $N_2(x_\varepsilon) \leftrightarrow N_1(x)$
end if

5.1 Merge interaction: does Phase 2 affect Phase 3?

A natural concern arises: if we merge $N_2(x)$ into $N_1(x)$ in Phase 2, does this affect our ability to merge $N_2(x_\varepsilon)$ in Phase 3?

The problem. In Algorithm 3, Phase 3 decides whether to merge $N_2(x_\varepsilon)$ by checking $\gamma^P_{\ell,i}$—the perturbation interval width, which bounds the gap $|\tilde{a}^{N_2}_{\ell,i} - a^{N_2}_{\ell,i}|$. But if Phase 2 *merged* $N_2(x)$ into $N_1(x)$ (i.e., set $a^{N_2}_{\ell,i} \leftarrow a^{N_1}_{\ell,i}$), then the variable $a^{N_2}_{\ell,i}$ no longer exists—it has been replaced by $a^{N_1}_{\ell,i}$. The Phase 3 constraint now links $\tilde{a}^{N_2}_{\ell,i}$ to $a^{N_1}_{\ell,i}$, *not* to the true $a^{N_2}_{\ell,i}$.

The relevant gap becomes:

$$|\tilde{a}^{N_2}_{\ell,i} - a^{N_1}_{\ell,i}| \;\le\; \underbrace{|\tilde{a}^{N_2}_{\ell,i} - a^{N_2}_{\ell,i}|}_{\le \gamma^P_{\ell,i}} + \underbrace{|a^{N_2}_{\ell,i} - a^{N_1}_{\ell,i}|}_{\le \gamma^T_{\ell,i}} \;=\; \gamma^C_{\ell,i}.$$

So after a Phase 2 merge, the effective interval width governing the Phase 3 decision is $\gamma^C = \gamma^T + \gamma^P$, *not* γ^P alone.

Proposition 8 (Merge interaction)**.** *Let $\theta > 0$ be the merge threshold. Suppose neuron (ℓ, i) satisfies:*

1. *$\gamma^T_{\ell,i} < \theta$ (mergeable in Phase 2), and*
2. *$\gamma^P_{\ell,i} < \theta$ (would be mergeable in Phase 3 without Phase 2 merge).*

After Phase 2 merges $N_2(x) \to N_1(x)$, the effective gap for Phase 3 becomes $\gamma^C_{\ell,i} = \gamma^T_{\ell,i} + \gamma^P_{\ell,i}$. If $\gamma^C_{\ell,i} \geq \theta$, then Phase 3 can no longer merge $N_2(x_\varepsilon)$, even though $\gamma^P_{\ell,i} < \theta$.

Proof. After Phase 2, $a^{N_2}_{\ell,i}$ is replaced by $a^{N_1}_{\ell,i}$. The Phase 3 LP relaxation or reuse requires $\tilde{a}^{N_2}_{\ell,i}$ to be within interval width $< \theta$ of its reference variable. The reference is now $a^{N_1}_{\ell,i}$, so the relevant width is:

$$I^C_{\ell,i,\text{up}} - I^C_{\ell,i,\text{down}} = \gamma^C_{\ell,i} = \gamma^T_{\ell,i} + \gamma^P_{\ell,i}.$$

By assumption $\gamma^T + \gamma^P \geq \theta$, so the merge condition fails. □

Example. Consider $\theta = 0.1$, $\gamma^T = 0.06$, $\gamma^P = 0.07$. Both are individually below θ, but $\gamma^C = 0.13 > \theta$. If we merge $N_2(x) \to N_1(x)$, we *cannot* also merge $N_2(x_\varepsilon) \to N_1(x)$.

Three resolution strategies.

1. **Strategy A: Independent decisions (Algorithm 3 as written).** Phase 2 checks γ^T, Phase 3 checks γ^P, each against θ independently. When Phase 2 merges $N_2(x) \to N_1(x)$ and Phase 3 merges $N_2(x_\varepsilon) \to N_2(x)$, the Phase 3 merge target (a^{N_2}) no longer exists—it was replaced by a^{N_1} in Phase 2. The Phase 3 interval constraint effectively uses the *composed* interval I^C as the gap from $N_1(x)$ (since the chain $N_2(x_\varepsilon) \to N_2(x) \to N_1(x)$ collapses).

   Correctness: This is still sound because the interval constraints are over-approximations. The Phase 3 constraint $a^{N_1} + I^P_{\text{down}} \leq \tilde{a}^{N_2} \leq a^{N_1} + I^P_{\text{up}}$ is *incorrect* (too tight) after Phase 2's merge. The correct constraint after Phase 2's merge should use I^C bounds: $a^{N_1} + I^C_{\text{down}} \leq \tilde{a}^{N_2} \leq a^{N_1} + I^C_{\text{up}}$.

   Fix: When both Phase 2 and Phase 3 merge, Phase 3's interval constraints must be updated to use I^C bounds, not I^P bounds.

2. **Strategy B: Greedy priority.** For each neuron, choose the *single most beneficial* merge:

   - If $\gamma^T < \gamma^P$: merge $N_2(x) \to N_1(x)$ (Phase 2); keep $N_2(x_\varepsilon)$ as a separate variable with interval constraint using I^C (since its reference is now a^{N_1}).
   - If $\gamma^P < \gamma^T$: keep $N_2(x)$ as separate; merge $N_2(x_\varepsilon) \to N_2(x)$ (Phase 3) with I^P bounds.
   - If $\gamma^C < \theta$: merge all three (Phase 2 + Phase 3).

   This avoids the interaction problem by performing at most one merge per neuron unless γ^C is small enough for both.

3. **Strategy C: Joint optimization.** Formulate the merge decision as a small combinatorial problem: for each neuron, decide which subset of $\{N_2(x), N_2(x_\varepsilon)\}$ to merge, maximizing binary variable savings subject to the constraint that all interval widths used are below θ. Formally, let $m_i^T, m_i^P \in \{0,1\}$ be merge indicators. The constraints are:

$$m_i^T = 1 \implies \gamma_i^T < \theta, \tag{33}$$

$$m_i^P = 1 \text{ and } m_i^T = 0 \implies \gamma_i^P < \theta, \tag{34}$$

$$m_i^P = 1 \text{ and } m_i^T = 1 \implies \gamma_i^C < \theta. \tag{35}$$

This is a simple per-neuron decision (no coupling across neurons), so it reduces to the following rule:

$$\text{For each neuron } i: \quad \begin{cases} m^T = 1, m^P = 1 & \text{if } \gamma^C < \theta, \\ m^T = 1, m^P = 0 & \text{if } \gamma^T < \theta \text{ and } \gamma^C \geq \theta, \\ m^T = 0, m^P = 1 & \text{if } \gamma^P < \theta \text{ and } \gamma^T \geq \theta, \\ m^T = 0, m^P = 0 & \text{otherwise.} \end{cases}$$

Recommended approach. Strategy C is the cleanest: it correctly accounts for the interaction, requires no post-hoc correction of interval bounds, and reduces to a simple per-neuron classification. We update Algorithm 3 accordingly:

Algorithm 3 Corrected hybrid variable management for neuron (ℓ, i)

Require: Interval widths γ^T, γ^P, γ^C; threshold θ

1: // **Phase 1: Encode** $N_1(x)$ — **always full big-M**
2: Create $a_{\ell,i}^{N_1}$ with binary and 4 constraints

3: // **Phase 2: Encode** $N_2(x)$
4: **if** $\gamma^T < \theta$ **then**
5: $\quad a_{\ell,i}^{N_2} \leftarrow a_{\ell,i}^{N_1}$ ▷ Merge (or LP relax)
6: $\quad$ `merged_N2` $\leftarrow$ **true**
7: **else**
8: $\quad$ Encode $a_{\ell,i}^{N_2}$ with full big-M
9: $\quad$ `merged_N2` $\leftarrow$ **false**
10: **end if**

11: // **Phase 3: Encode** $N_2(x_\varepsilon)$
12: $\gamma_{\text{eff}} \leftarrow$ **if** `merged_N2` **then** γ^C **else** γ^P ▷ Key correction
13: **if** $\gamma_{\text{eff}} < \theta$ **then**
14: $\quad \tilde{a}_{\ell,i}^{N_2} \leftarrow$ reference variable ▷ a^{N_1} if merged, a^{N_2} otherwise
15: $\quad$ Use I^C bounds if `merged_N2`, else I^P bounds
16: **else**
17: $\quad$ Encode $\tilde{a}_{\ell,i}^{N_2}$ with full big-M
18: $\quad$ Link to reference with appropriate interval constraints
19: **end if**

The critical insight is line 13: when $N_2(x)$ was merged into $N_1(x)$, the effective gap for the $N_2(x_\varepsilon)$ decision is γ^C, not γ^P. This ensures soundness while still maximizing variable savings.

6 Idea 5: Difference-Variable Formulation

Instead of encoding $a_{\ell,i}^{N_2}$ and $\tilde{a}_{\ell,i}^{N_2}$ as independent variables, reparametrize in terms of *differences*:

$$\delta_{\ell,i}^{T} = a_{\ell,i}^{N_2} - a_{\ell,i}^{N_1}, \qquad \delta_{\ell,i}^{T} \in [I_{\text{down}}^{T},\ I_{\text{up}}^{T}], \tag{36}$$

$$\delta_{\ell,i}^{P} = \tilde{a}_{\ell,i}^{N_2} - a_{\ell,i}^{N_2}, \qquad \delta_{\ell,i}^{P} \in [I_{\text{down}}^{P},\ I_{\text{up}}^{P}]. \tag{37}$$

Then the network outputs become:

$$v_{\text{out}}^{N_2} = f_{N_1}(x) + g^{T}(\delta_1^{T}, \ldots, \delta_L^{T}), \tag{38}$$

$$v_{\text{out}}^{N_2,p} = f_{N_1}(x) + g^{T}(\delta^{T}) + g^{P}(\delta^{P}), \tag{39}$$

where g^T and g^P capture the cumulative effect of per-layer differences.

Advantage. The difference variables δ^T, δ^P have *tight, known bounds* from the interval propagation. When γ^T or γ^P is small, δ is confined to a small range, which:

1. Gives the solver tighter variable bounds (faster bound tightening).
2. When $\delta = 0$ is the only feasible value, Gurobi can fix the variable in presolve (effectively achieving variable elimination automatically).
3. Avoids the need for separate binary variables for the N_2 copies when the difference is bounded tightly enough that the ReLU behavior is determined by N_1's binary alone.

ReLU on the difference. The challenge is encoding $\sigma(z^{N_1} + \Delta z)$ where $\Delta z = (W_2 - W_1)^{\top} a_{\text{prev}}^{N_1} + W_2^{\top} \delta_{\text{prev}}^{T}$. When z^{N_1} is already encoded with binary α:

- If $\alpha = 1$ (active): $\sigma(z^{N_1} + \Delta z) = z^{N_1} + \Delta z$ provided $z^{N_1} + \Delta z \geq 0$ (checkable from bounds).
- If $\alpha = 0$ (inactive): $\sigma(z^{N_1} + \Delta z) = 0$ provided $z^{N_1} + \Delta z \leq 0$.
- Otherwise: a new binary is still needed, but the tighter bounds on Δz often resolve the case.

7 Implementation Roadmap

7.1 Step 1: Compute all three interval types

Before encoding the MIP, run three propagations:

1. `propagate_intervals(nn1, nn2, "org")` $\rightarrow I^T$
2. `propagate_pert_intervals(nn2, ε)` $\rightarrow I^P$
3. Propagate I^C directly via Algorithm 1, or approximate as $I^C \approx I^T + I^P$ (Proposition 6)

7.2 Step 2: Classify each neuron

For each (ℓ, i), assign a *mode*:

Mode	Condition	Encoding
Full	$\gamma^T \geq \theta$ and $\gamma^P \geq \theta$	3 separate big-M encodings
MergeN2	$\gamma^T < \theta$ (or $\gamma^T = 0$)	Reuse/relax $N_2(x)$; keep $N_2(x_\varepsilon)$
MergeN2p	$\gamma^P < \theta$ (or $\gamma^P = 0$)	Keep $N_2(x)$; reuse/relax $N_2(x_\varepsilon)$
MergeAll	$\gamma^C < \theta$	Only $N_1(x)$ is encoded; rest via offsets

7.3 Step 3: Modified encoding pass

Modify `get_perturbation_specific_keys_linf_transfer` to:

1. Encode $N_1(x)$ fully (as today).
2. For $N_2(x)$: check $\gamma^T_{\ell,i}$ per neuron. In MergeN2 or MergeAll mode, skip variable creation and wire the output of $N_1(x)$'s ReLU directly (possibly with an affine correction $+\bar{I}^T$).
3. For $N_2(x_\varepsilon)$: check $\gamma^P_{\ell,i}$ or $\gamma^C_{\ell,i}$. In merge modes, wire the appropriate reference variable.
4. Add composed interval constraints (17) for non-merged neurons as additional tightening.

7.4 Step 4: Soundness

- **Exact reuse** ($\gamma = 0$ or $\phi_{\text{dep}} = 0$): Sound by construction.
- **LP relaxation** ($0 < \gamma < \theta$): Sound—the LP relaxation is a valid outer approximation of the ReLU feasible set. The solution is still a valid upper bound on δ_{diff}.
- **Midpoint substitution** (Section 3): *Not sound* in general. Use only for heuristic speedup or as a warm-start strategy, not for the final proof.

8 Expected Impact

For a 4×10 network (4 hidden layers, 10 neurons each):

- Current transfer MIP: up to $3 \times 4 \times 10 = 120$ binary variables (3 copies × 4 layers × 10 neurons).
- With full three-way merge on neurons where $\gamma^C < \theta$: reduced to 40+ (number of non-merged neurons) ×2 binaries.
- For similar networks ($W_1 \approx W_2$) and small ε, many neurons will qualify for merging, potentially cutting the binary count by 40–60%.

The composed interval I^C also provides *direct* tightening between the first and last network copies, which can accelerate Gurobi's node processing even when variables are not merged.